

CSCE 313: Introduction to Computer Systems

UNIX File I/O

Background

Applications running in the system are address space abstractions called **processes**.

Processes need to talk to the outside world and they do so via privileged interface (**system calls**, **interrupts**).

In this module we will focus on the the UNIX I/O interface.

- Transfer happens in **bytes** and using **buffers**
- Every I/O device is treated as a **file** for uniformity
- Basic operations are **read** and **rite**
- Checks and Balances are needed to ensure an orderly and efficient transfer. These are done through **open** and **close**, and in the form of session enabled with **file descriptors**.

Outline

1. UNIX I/O
2. Common operations (system calls)
 - `read()` and `write()` for basic read and write from/to an I/O device
 - `open()` and `close()` to enable checks and balances
3. File Descriptors (FD)

What is I/O?

I/O is the process of **copying data between the main memory and external devices** (disk drives, terminals, networks, etc.).

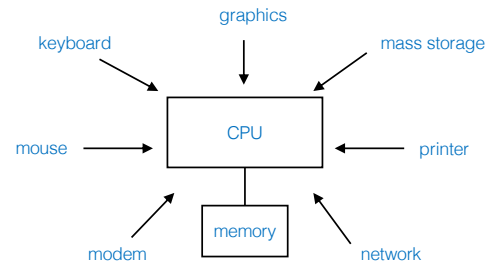
Language run-time systems provide higher-level facilities for performing I/O (e.g. `printf/cout`, `scanf/cin`).

- On Unix systems, these higher-level I/O functions eventually call **System-Level Unix I/O functions** provided by the Kernel (e.g., `printf()` internally calls `write()`)
- The same applies to Windows and other OSs
- `printf()` calls native windows function `WriteFile()`

Unix Files

A Unix file is a sequence of m bytes:

$B_0, B_1, \dots, B_k, \dots, B_{m-1}$



All I/O devices (networks, disks, terminals) are represented as files:

- **`/dev/sda2`** (**`/usr`** disk partition; `a` is the order and `#2` is the partition)
- **`/dev/tty2`** (terminal)

Even the kernel is represented as a file:

- **`/dev/kmem`** (kernel memory image)
- **`/proc`** (kernel data structures)

All I/O is performed by reading and writing files

Common Unix File Types

- **Regular files**

File containing user/app data (binary, text, whatever)

- **Directory files**

A file that contains the names and locations of other files

- **Character special and block special files**

Terminals (character special) and disks (block special)

- **FIFO** (named pipe)

A file type used for inter-process communication

- **Socket**

A file type used for network communication between processes

Unix I/O

Basic Unix I/O operations (system calls):

– Opening and closing files

- **open()** and **close()**

– Reading and writing a file

- **read()** and **write()**

– Current file position

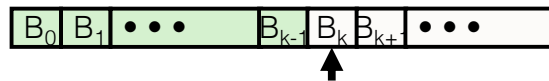
- Kernel maintains a file position (initially 0) for each open file (aka **file pointer**)

- Indicates next (byte) offset into file to read or write

- Reading and writing automatically **advance** the file pointer

- **lseek()** can be used to change file position **manually** at programmer's wish

– Instead of the default which is tied to read and write operations



Current file position = k

Opening Files

Opening a file informs the kernel that we are ready to access that file.

```
int fd = open("/etc/hosts", O_RDONLY);
if (fd < 0) {
    perror("open");
    exit(1);
}
```

This code returns an identifying integer **file descriptor**

fd == -1 indicates that an error occurred

The kernel keeps track of all information about the open file. e.g., current file position, permissions etc.

Opening Files

Each process created by a Unix shell begins with three open files associated with the terminal:

- 0: standard input
- 1: standard output
- 2: standard error

On success, open returns the smallest available fd value

e.g., in the below picture **fd = 3** (0-2 are occupied)

```
int fd = open("/etc/hosts", O_RDONLY);
if (fd < 0) {
    perror("open");
    exit(1);
}
```

Closing Files

Closing a file informs the kernel that we are finished accessing that file

```
if ((retval = close(fd)) < 0) {
    perror("close");
    exit(1);
}
```

The kernel responds by

- freeing some data structures
- restoring the descriptor to the pool of available descriptors

When a process terminates for any reason, the kernel performs **close** on all open files

Creating a File

So far, we have only opened files for reading. The file must have existed beforehand.

To create a file, we call `open()` with **flag** set to write and **mode** specifying appropriate **permissions**.

```
int open(const char *pathname, int flags, mode_t mode);
```

Consult the [linux man pages](#) for more details.

However, here are some common use cases.

We will commonly use one of these 3 flags: `O_RDONLY`, `O_WRONLY`, or `O_RDWR` for read, write or both, respectively.

There are other sub-flags that can be bitwise OR-ed with these.

Note: When opening for read, no need to use the last argument mode, because you are not creating a file.

However, when creating one, you must set the permissions of a file, which is next.

Creating a File

```
int open(const char *pathname, int flags, mode_t mode);
```

The following two items specifies permissions of a file:

Types of Access	Types of Users
• Read (R) • Write (W) • Execute (X)	• User/Owner (U) • Group (G) • Others (O)

For each user, there is a 3-digit octet **RWX** that determines what level of permissions he has.

For instance, if the User/Owner has R, W, and X permission, he gets 7 (=111 in binary)

if the group has only R and W permissions, everybody in that group gets 6 (=110 in binary), and no permission for others would give 0. Together, the permission for the file should be: **760**

Creating a File

```
int open(const char *pathname, int flags, mode_t mode);
```

To create a file only for yourself:

```
int main (){
    int fd = open ("test.txt", O_WRONLY|O_CREAT|O_TRUNC, S_IRWXU);
    if (fd<0){
        perror ("Cannot create file ");exit(0);
    }
    cout << "File created successfully" << endl;
}
```

```
prompt> ls -l
total 36
-rwxrwxr-x 1 osboxes osboxes 17424 Sep  7 15:23 a.out
-rw-rw-r-- 1 osboxes osboxes  256 Sep  2 14:21 exec1.cpp
-rw-rw-r-- 1 osboxes osboxes  313 Sep  7 15:23 fileio1.cpp
-rw-rw-r-- 1 osboxes osboxes  288 Sep  1 23:08 fork1.cpp
-rw-rw-r-- 1 osboxes osboxes  496 Sep  2 14:01 fork2.cpp
-rwx----- 1 osboxes osboxes   0 Sep  7 15:23 test.txt
```

Creating a File

```
int open(const char *pathname, int flags, mode_t mode);
```

However, keep in mind that **you cannot just directly put 700** for permission because 700 in decimal is not the same as 700 in octet. You can write 0700 though, because that represents octal number.

```
int main (){
    int fd = open ("test.txt", O_WRONLY|O_CREAT|O_TRUNC,700); // this is incorrect
    if (fd<0){
        perror ("Cannot create file ");exit(0);
    }
    cout << "File created successfully" << endl;
}
```

```
prompt> ls -l
total 36
-rwxrwxr-x 1 osboxes osboxes 17424 Sep  7 15:22 a.out
-rw-rw-r-- 1 osboxes osboxes  256 Sep  2 14:21 exec1.cpp
-rw-rw-r-- 1 osboxes osboxes  309 Sep  7 15:22 fileio1.cpp
-rw-rw-r-- 1 osboxes osboxes  288 Sep  1 23:08 fork1.cpp
-rw-rw-r-- 1 osboxes osboxes  496 Sep  2 14:01 fork2.cpp
-rw-rw-r-- 1 osboxes osboxes   0 Sep  7 15:22 test.txt
```

Reading Files

Reading a file copies bytes from the current file position to memory, and then updates file position.

```
char buf[512];
int fd = open (filename, O_RDONLY);
int nbytes;
/* Then read up to 512 bytes from file fd */
if ((nbytes = read(fd, buf, sizeof(buf))) < 0) {
    perror("read");
    exit(1);
}
```

Returns number of bytes read from file `fd` into `buf`

- Return type **ssize_t** is signed integer
- **nbytes < 0** indicates that an error occurred
- **Short counts** (**nbytes < sizeof(buf)**) are possible and are not errors (eg. EOF when the file does not have `nbytes` of data starting from the file pointer, reading text from terminal where user gave < `nbytes` etc.)

Reading files: example

Output of the following program reading a file containing “foobar”

```
char buf [2];
int fd = open (filename, O_RDONLY);
int nbytes = read(fd, buf, sizeof(buf)); // buf = ?
nbytes = read(fd, buf, sizeof(buf)); // buf = ?
```

Answer: “fo” and “ob”

Why not “fo” and “foob”??

Writing files

Writing a file **writes** bytes from memory to the current file position, and then **updates** current file position

```
char buf[512];
int nbytes; /* number of bytes written*/
int fd = open (filename, O_WRONLY|O_CREAT, 0700);
/* Then write up to 512 bytes from buf to file fd */
if ((nbytes = write(fd, buf, sizeof(buf)) < 0) {
    perror("write");
    exit(1);
}
```

Returns number of bytes written from `buf` to file `fd`

- **nbytes < 0** indicates that an error occurred
- As with reads, short counts are possible and are not errors!

Writing files: example

What would be the content of the file after executing the following program?

```
char buf[] = {'a', 'b'}; // what if char* buf = "ab"
int fd; /* file descriptor */
/* Open the file fd ... */
/* Then write up to 2 bytes from buf to file fd */
write(fd, buf, sizeof(buf)); //file = ?
write(fd, buf, sizeof(buf)); //file = ?
```

abab

File Representation: Three Tables

Descriptor Table

- Each process has its own separate descriptor table (only accessible directly by the kernel)
- Entries are indexed by the process's open file descriptors
- Each entry points to a **File Table** entry

File Table

- Shared by all processes.
- Each entry consists of
 - **File Cursor**
 - **Reference count**: # of descriptor entries from all processes that point to it
 - ptr to the v-node table entry, along with status flags

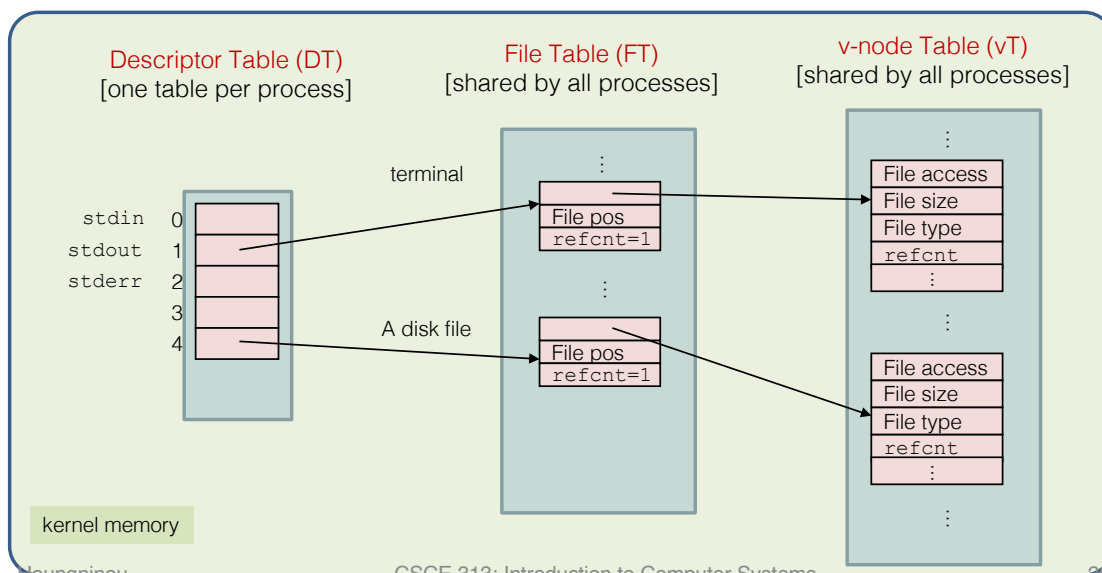
v-node Table

- Each entry (1 per file) contains information about the file meta data (e.g., access/modification date, permissions)

How the Unix kernel represents open files?

Two descriptors referencing two distinct open disk files.

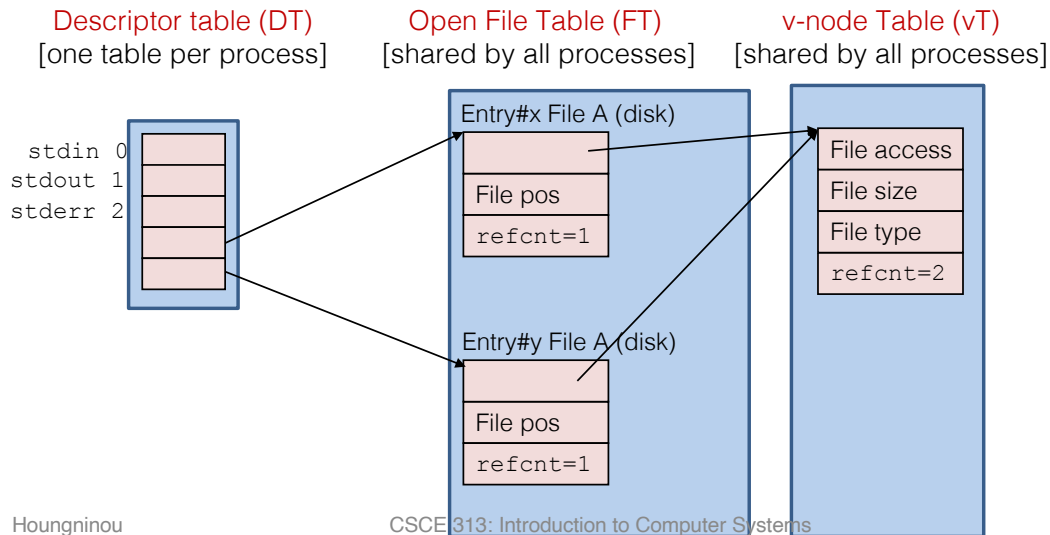
Descriptor 1 (stdout) points to terminal, and descriptor 4 points to an open disk file.



File Sharing

Two distinct DTs sharing the same disk file through two distinct open FT entries

- E.g., calling **open** twice with the same **filename** argument
- Every **open** creates a new DT and a new FT entry, also possibly a vT entry



Example

Suppose the disk file foobar.txt consists of the six ASCII characters “foobar”. Then what is the output of the following program:

```
int main (){
    char c;
    int fd1 = open("foobar.txt", O_RDONLY, 0);
    int fd2 = open("foobar.txt", O_RDONLY, 0);
    read(fd1, &c, 1);
    cout << c << endl;
    read(fd2, &c, 1);
    cout << c << endl;
}
```

Answer: The descriptors fd1 and fd2 each have their own open file table entry so each descriptor has its own file position for foobar.txt.

Thus the read from fd1 reads the first byte of foobar.txt and the output is “f”. Same is true for fd2 and the output is again “f”